Formula 1 Race Management & Telemetry-Inspired Performance Analytics System

Project Synopsis

UCS310 – Database Management Systems

B.E. 2nd Year – COE

THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

Department of Computer Science and Engineering
Thapar Institute of Engineering and Technology
Patiala - 147001

Submitted By :

Sukhansh Mittal (1024030318)
Vikramaditya Singh (1024030315)

Ishjaap Singh (1024030327)

Submitted To :

Diksha Arora

1. Introduction

Formula 1 (F1) is one of the most data-driven sports in the world, where race strategies and performance decisions rely heavily on data analysis. This project focuses on designing a relational database system to manage official race data and provide a telemetry-inspired analytics layer for performance evaluation.

The motivation behind selecting this problem is the growing importance of data analytics in motorsport and the opportunity to apply DBMS concepts to a real-world inspired domain. A Database Management System is preferred over a file-based system because it ensures structured storage, faster querying, data integrity, and reduced redundancy.

The system models seasons, circuits, teams, drivers, races, and results, and also uses synthetic telemetry to analyze driver performance at lap and sector levels.

Telemetry data will be synthetically generated using Python scripts based on realistic lap timing ranges.

2. Problem Statement

Managing and analyzing Formula 1 race data manually or using basic files is inefficient and error-prone. Real F1 data is complex and interconnected, involving multiple factors such as drivers, teams, circuits, and race statistics.

Limitations of existing/manual systems:

- Data redundancy and inconsistency.

- Impossible to perform complex analysis.

- Poor scalability.

- Lack of structured relationships

This project addresses these issues by creating a structured DBMS that stores and analyzes F1 race and performance data.

3. Objectives of the Project

- To design a database using ER data modeling for F1 data
- To convert the ER model into one or more relational tables
- To apply normalization up to 3NF.
- To implement the database using SQL
- To use PL/SQL constructs such as procedures, triggers, and functions
- To ensure data consistency using constraints and transactions
- To support performance analytics like lap and sector comparisons

4. Scope of the Project

Functional boundaries:

- Storage of race, driver, team, and season data
- Synthetic telemetry data generation
- Analytical querying for performance insights

Types of users:

- Admin
- Analyst/Researcher

Modules:

- Race Management Module
- Telemetry Simulation Module
- Performance Analytics Module

5. Proposed System Description

The proposed system is a relational DBMS that stores official F1 race data and integrates a synthetic telemetry layer. Since real telemetry is proprietary, synthetic but realistic data is generated for academic analysis.

Key features:

- Storage of seasons, circuits, teams, drivers, races, and results.

- Lap times and sector times tracking.

- Tyre usage data storage and comparison.

- Track condition based performance querying.

The system improves efficiency, consistency, and integrity by enforcing relationships, constraints, and structured querying.

6. Database Design

6.1 Entity–Relationship (ER) Diagram

Data:

- Season

- Circuit

- Team

- Driver

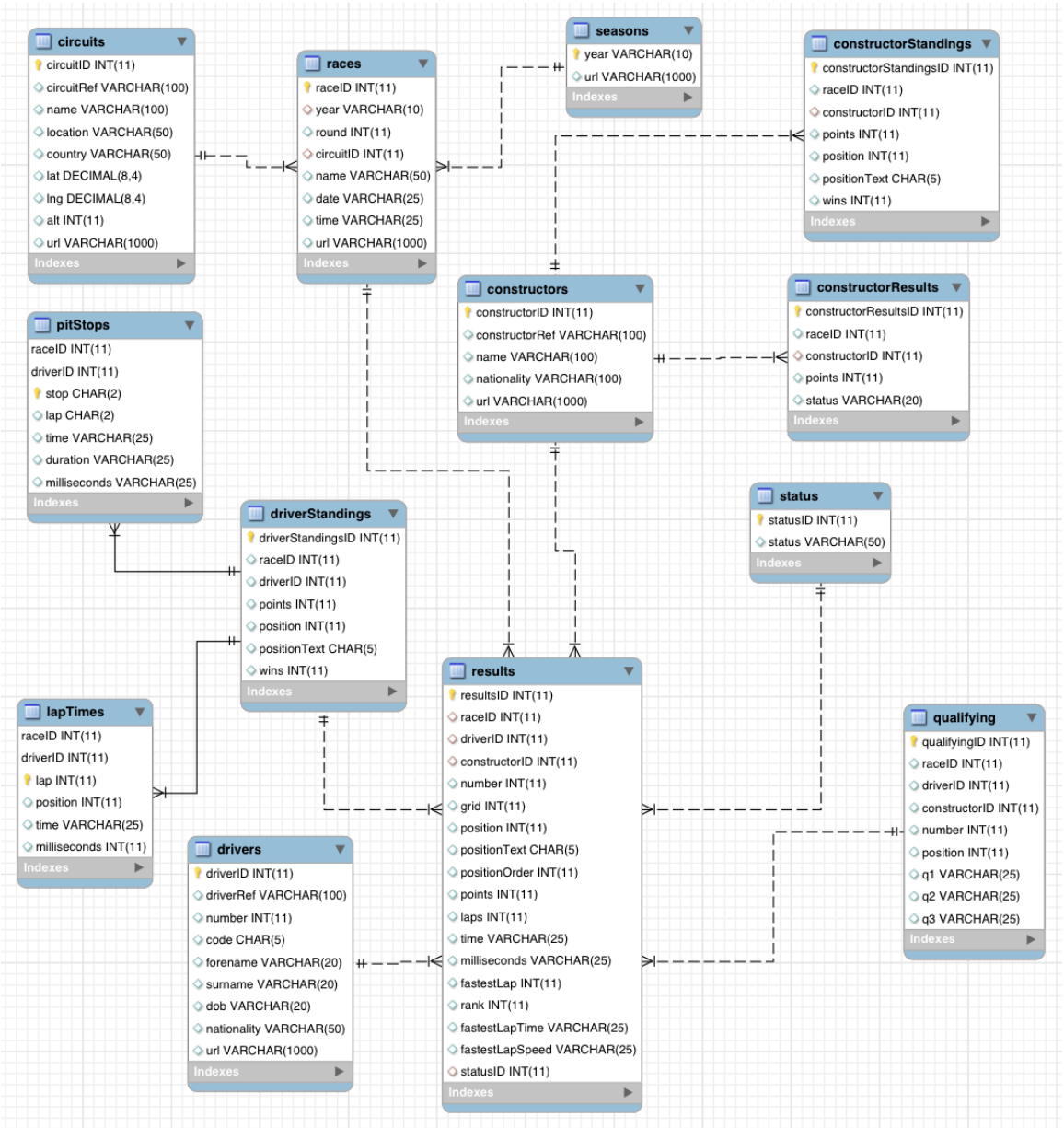
- Race

- Result

- LapData

- TyreData

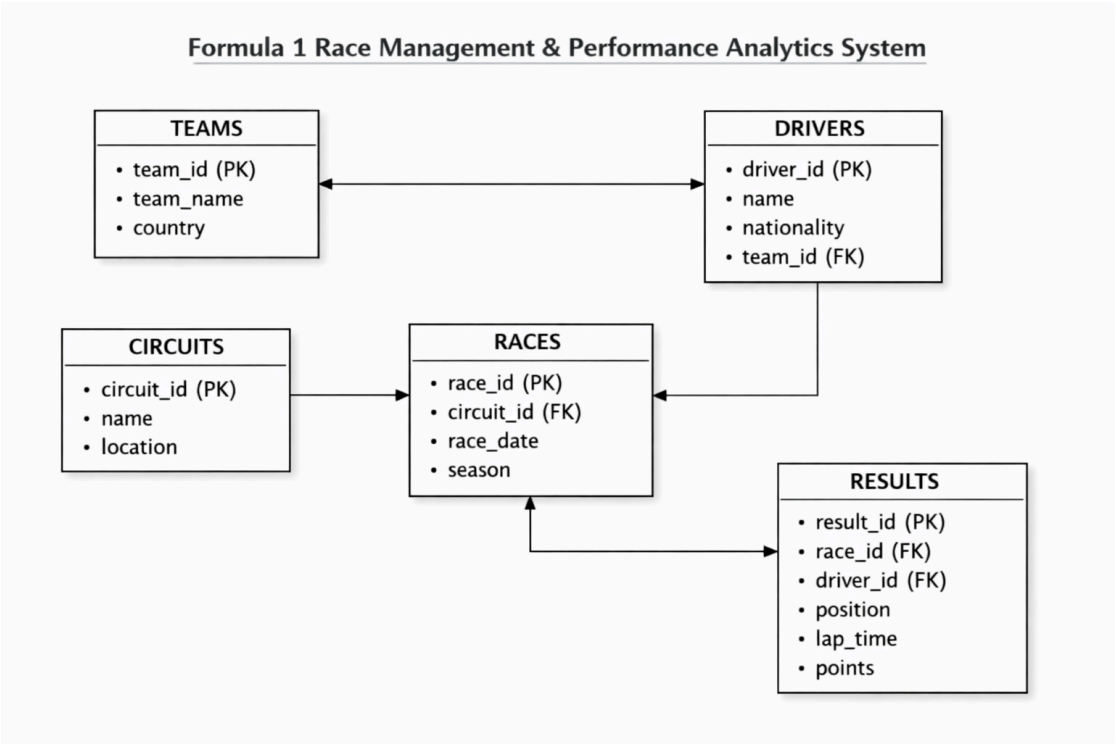
These entities are interconnected to maintain referential integrity. The ER diagram (attached below) visually represents these relationships and their cardinality.



6.2 Relational Schema

The following tables define the structure of the racing database:

- **SEASON**
  - `SeasonID` (Primary Key)
  - `Year`
- **CIRCUIT**
  - `CircuitID` (Primary Key)
  - `Name`
  - `Location`
- **TEAM**
  - `TeamID` (Primary Key)
  - `TeamName`
- **DRIVER**
  - `DriverID` (Primary Key)
  - `Name`
  - `TeamID` (Foreign Key)
- **RACE**
  - `RaceID` (Primary Key)
  - `SeasonID` (Foreign Key)
  - `CircuitID` (Foreign Key)
  - `Date`
- **RESULT**
  - `ResultID` (Primary Key)
  - `RaceID` (Foreign Key)
  - `DriverID` (Foreign Key)
  - `Position`
- **LAPDATA**
  - `LapID` (Primary Key)
  - `RaceID` (Foreign Key)
  - `DriverID` (Foreign Key)



7. Normalization

Functional dependencies will be identified and decomposition will be demonstrated step-by-step from 1NF to 3NF/BCNF.The process will involve identifying functional dependencies and then demonstrating the step-by-step decomposition from First Normal Form (1NF) up to Third Normal Form (3NF) or Boyce-Codd Normal Form (BCNF).

8. Database Implementation

8.1 SQL Implementation

The SQL component includes the creation of tables using DDL commands (CREATE, ALTER), data manipulation using DML commands (INSERT, UPDATE, DELETE), and querying using SELECT statements with joins, subqueries, and aggregate functions. Views are created for simplified data retrieval and reporting.

8.2 PL/SQL Components

The backend incorporates PL/SQL constructs for automation and data integrity, including:
- Procedures
- Functions
- Triggers
- Cursors
- Exception Handling

9. Transaction Management & Concurrency
- Using COMMIT, ROLLBACK, SAVEPOINT
- ACID properties
- Basic concurrency control

10. Tools & Technologies Used

- Database: Oracle
- Programming Language: Python
- Interface Tools: SQL Developer
- SQL and PL/SQL for backend implementation

11. Expected Outcomes

- A fully normalized database.
- Efficient data retrieval using optimized queries and indexing.
- Improved data accuracy and consistency.
- Automation using triggers and procedures.
- IReduced query execution time for race result analysis.